A REPORT

ON

HYBRID QUANTUM-CLASSICAL COMPUTATION

BY

Ameya Ajay Thete	2018B5A70885G
Mankrit Singh	2018A7PS0192P
Sanath Salil	2018B4A70812H

AT

UST Global, Thiruvananthapuram

A Practice School-I Station of

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

(June, 2020)

A REPORT

ON

HYBRID QUANTUM-CLASSICAL COMPUTATION

BY

Ameya Ajay Thete	2018B5A70885G	M.Sc. Physics and B.E. Computer Science
Mankrit Singh	2018A7PS0192P	B.E. Computer Science
Sanath Salil	2018B4A70812H	M.Sc. Mathematics and B.E. Computer Science

Prepared in partial fulfilment of the

Practice School-I Course Nos.

BITS F221/BITS F231/BITS F241

AT

UST Global, Thiruvananthapuram

A Practice School-I Station of

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

(June, 2020)

Acknowledgements

We would like to extend our heartfelt gratitude to our Practice School faculty mentor, Prof. G Geethakumari, BITS Pilani - Hyderabad Campus; and our industry mentor Mr. Simsar Vengasseri, Infinity Labs, UST Global, Thiruvananthapuram, for their constant and undivided attention, help and support during the course of this project.

We would also like to thank the Practice School Division, BITS Pilani, for accommodating the Practice School program and providing us an opportunity to work in the industry even during the unprecedented pandemic arising due to COVID-19, without which we would be deprived of the professional and research experience as well as the industry exposure which the Practice School program aims to provided.

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI
(RAJASTHAN)
Practice School Division

Station: UST Global, Thiruvananthapuram
Duration: 18 May 2020 - 27 June 2020 **Date of Start:** 18 May 2020
Date of Submission: 24 June 2020

Title of the Project: Hybrid Quantum-Classical Computation

ID No./Name(s)/Discipline(s) of the student(s):

Ameya Ajay Thete	2018B5A70885G	M.Sc. Physics and B.E. Computer Science
Mankrit Singh	2018A7PS0192P	B.E. Computer Science
Sanath Salil	2018B4A70812H	M.Sc. Mathematics and B.E. Computer Science

Name(s) and designation(s) of the expert(s): Simsar Vengasseri, Infinity Labs, UST Global

Name(s) of the PS Faculty: G Geethakumari, BITS Pilani - Hyderabad Campus

Key Words: Quantum Computing, Machine Learning, Quantum Optimisation, Quantum Machine Learning, Bioinformatics

Project Areas: Quantum Computing, Quantum Machine Learning, Biological Applications of Machine Learning

Abstract: There exists a bottleneck in conventional high-performance computing that is rapidly approaching. One upcoming area to ameliorate computation is quantum computation. Over the past few years, machine learning strategies have also shown a promising application in biological sciences. Inspired by advances in these fields, we wish to evaluate the performance of a well known quantum machine learning algorithm — the quantum support vector machine — on multiomic human cancer data from the Cancer Genome Atlas. We would also like to compare this performance with quantum annealing as well as some classical machine learning algorithms for assessment.

Table of Contents

1 Introduction **2**

1.1 Qubits . . . 2

1.2 Quantum Machine Learning Algorithms . . . 2

1.3 Our Project . . . 3

2 Dataset **4**

2.1 Whole Exam Sequencing . . . 4

2.2 RNA-Seq . . . 4

2.3 miRNA-Seq . . . 4

2.4 DNA Methylation Arrays . . . 4

2.5 Genotyping Arrays (CNV) . . . 4

3 Principal Component Analysis **6**

3.1 Principal Components . . . 6

4 Feature Scaling **7**

5 Quantum Support Vector Machine **8**

5.1 Least Squares SVM . . . 8

5.2 Working of QSVM . . . 9

6 Methods and Experiments **10**

7 Results and Analysis **10**

8 Bibliography **13**

9 Glossary **14**

1 Introduction

1.1 Qubits

In classical computational theory, the fundamental unit of information is a bit, which can take the value 0 or 1, corresponding to a high or low voltage signal. A register is a device which can be thought of as a string of bits. Due to the deterministic nature of a classical bit, any such classical register can be read with a hundred percent certainty of the value it contains. In the realm of quantum computation, a the fundamental unit of information is called the quantum bit — or 'qubit' for short. A qubit is the simplest two-level quantum system. For example, a photon's polarisation or an electron's spin state are the most common examples of a qubit. The electron can exist either in a 'spin-up' state (corresponding to a classical 0), or a 'spin-down' state (corresponding to a classical 1). However, a quantum mechanics allows this qubit to exist in a coherent superposition of both these states, and this forms the fundamental basis of the theory of quantum computation [1, 2].
Mathematically, a qubit is an element of a projective Hilbert space and is expressed as a normalised linear combination of the orthonormal basis vectors of this space. For the Hilbert space $\mathbb{C}^2$, the basis is $\mathcal{B} = \{|0\rangle, |1\rangle\}$ where $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ The Dirac notation is commonly used to represent qubits. Therefore, the quantum mechanical state of a qubit, $|\psi\rangle$ is

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle \tag{1}$$

Where α and β are probability amplitudes and are both complex numbers. According to the Born rule, the absolute-squared sum of these is 1, *id est*, $|\alpha|^2 + |\beta|^2 = 1$

Geometrically, qubits are represented by a Bloch sphere — a unit 2-sphere where the antipodal points correspond to the orthogonal basis vectors of the Hilbert space. Here, (1) can be expressed as

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle \tag{2}$$

On measurement however, the state $|\psi\rangle$ collapses to either $|0\rangle$ with probability $|\alpha|^2$ or $|1\rangle$ with probability $|\beta|^2$.

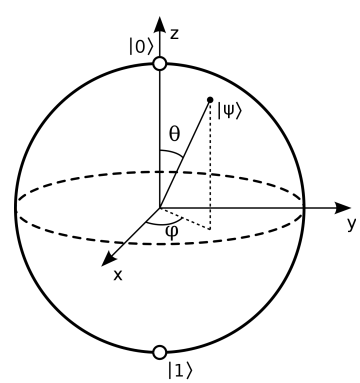

Figure 1: Representation of a qubit on a Bloch sphere

1.2 Quantum Machine Learning Algorithms

A quantum algorithm is an algorithm that is designed to run on a quantum computer, or a model of one. Similar to a classical algorithm, it describes a set of steps that are sequentially performed on a quantum computer (usually using a circuit-based model of computation). Due to effects such as superposition or entanglement, quantum algorithms provide a significant speedup compared to their classical counterparts on some problems, but a problem that is undecidable (such as the halting problem, remains as such even on a quantum computer) [2]. Algorithms based on the gate model are the most common type of quantum algorithm. Here, the algorithm

involves applying certain quantum gates (a representation of an unitary transformation) onto a register of qubits which is then measured. This measurement followed by some classical post-processing, yields the answer to the problem.

Quantum machine learning is an emerging interdisciplinary area wherein machine learning algorithms are executed on a quantum computer, usually on classical data. Quantum-enhanced machine learning often improves and expedites classical machine learning techniques, due to the exponential speedup provided by using quantum effects and by simulating quantum states (also known as an ansatz). This typically requires one to encode the given classical data set into a quantum computer to make it accessible for quantum information processing. Subsequently, quantum information processing routines are applied and the result of the quantum computation is read out by measuring the quantum system. For example, the outcome of the measurement of a qubit reveals the result of a binary classification task. Hybrid Quantum Algorithms usually involve a classical part of computation which performs typical gradient descent like computations while a quantum state is simulated based on the output of the classical computation. The output of a measurement on this quantum is then used to provide feedback on the classical part of the algorithm. This is done until we converge to an acceptable cost value.

At the time of writing this report, there are few machine learning algorithms that are easily accessible or have a respectable amount of research done into them so that they can be commercially utilised. Some of these are Quantum annealing, Quantum neural networks, Quantum Support Vector Machines, and Quantum process tomography. We found that many of the listed quantum machine learning algorithms did not have reproducible code that we could implement, therefore we decided to implement the quantum support vector machine, which has been implemented in IBM's Qiskit-Aqua framework.

1.3 Our Project

In our project, we wish to leverage these quantum machine learning algorithms, particularly the Quantum Support Vector Machine (QSVM) and study its relevance to biological problems. We aim to present a performance comparison of machine learning algorithms executed on classical and quantum computers. We aim to present a performance comparison of machine learning algorithms executed on classical and quantum computers. The motivation for this arose from a paper that has evoked a good amount of discussion – 'Quantum Annealing versus Classical Machine Learning applied to a simplified computational biology problem' [3]. This paper compares the efficiency and accuracy of the Quantum Annealing process versus traditional classical machine learning processes. The results were promising. The quantum process was worse than the classical processes in most cases. It was comparable on a binary classification task and performed much better than classical algorithms when the datasets were reduced. We thought of extending on the work in this paper by running the QSVM on the same datasets to gauge its performance versus annealing process and the classical algorithms. There are also very few (if any) comparisons between two Quantum Processes which served as further motivation for this project.

This report illustrates the algorithm we have selected, the dataset we chose, the procedure we followed and the results we obtained.

2 Dataset

The dataset we chose to implement our machine learning algorithms is a high-dimensional, multi-comics dataset compiled by Richard Li et al. from The Cancer Genome Atlas. It is a binary classification dataset for two types of breast cancers — Luminal A and Luminal B. Whole exam sequencing, DNA Methylation Array, RNA, miRNA and Genotyping RNA data (CNV) were derived from the Genome Data Commons (GDC) portal. The method for compiling the dataset from each data type is described below. This procedure was carried out by Richard Li et al [3]. and is mentioned here for the sake of completeness.

2.1 Whole Exam Sequencing

GDC harmonised level 2 Variant Cell Format (VCF) files were retrieved. These were converted to Genomically Ordered Relational (GOR) database formats. DeepCODE scores were calculated for all variants and the variants were kept if their DeepCODE score was greater than 0.51. Call copies for each variant was mapped to its given gene and the counts of all the variants ascribed to a given gene were added together into a single count value.

2.2 RNA-Seq

GDC harmonised level-3 mRNA quantification data was retrieved as unnormalised raw read counts from HT-Seq. Raw mapping counts were combined into a count matrix with genes as rows and counts as samples and normalised using the trimmed mean of M-values (TMM). Lowly expressed genes were filtered out by requiring read counts to be greater that 1 per million reads for more than 10% of the samples.

2.3 miRNA-Seq

GDC harmonised level-3 mi-RNA quantification data was retrieved as unnormalised raw read counts from miRNA profiling pipeline. miRNAs were filtered by retaining only experimentally validated gene targets from the miRBase reference. Raw mapping counts were combined into a count matrix with genes as rows and counts as samples and normalised using the trimmed mean of M-values (TMM). Lowly expressed genes were filtered out by requiring read counts to be greater that 1 per million reads for more than 10% of the samples.

2.4 DNA Methylation Arrays

GCD harmonised level-3 beta values derived from Illumina Infinium Human Methylatiomn27 and HumanMethylation450 arrays was retrieved. NA and zero beta values were replaced with the minimum beta value across all probes and all sales in each batch. Beta values of 1 were replaced with th maximum beta value less than 1 across all probes and all samples in each batch. This value was converted to M values using

$$M(\beta) = \log_2 \left(\frac{\beta}{1-\beta} \right) \tag{3}$$

2.5 Genotyping Arrays (CNV)

GISTIC2 processed copy number variation (CNV) data was retrieved from cBioPortal. CNV data was compiled into a matrix with samples as rows and genes as columns and all NA values were removed. Thr segment means

were converted to linear copy numbers and mapped to gene symbols using

$$\text{Linear Copy Number} = 2 \times 2^{\text{Segment Mean}} \tag{4}$$

This data from each of the five genomic data types was then concatenated into a single data matrix, with samples represented as rows and genes as columns. Then, each feature was scaled to zero mean and unit variance (using Scikit-Learn's `StandardScaler` method) and the mean and variance from the training sets were then used to standardise the test sets. Finally we obtained the Luminal A vs Luminal B dataset. It had 311 samples and 66,801 features.

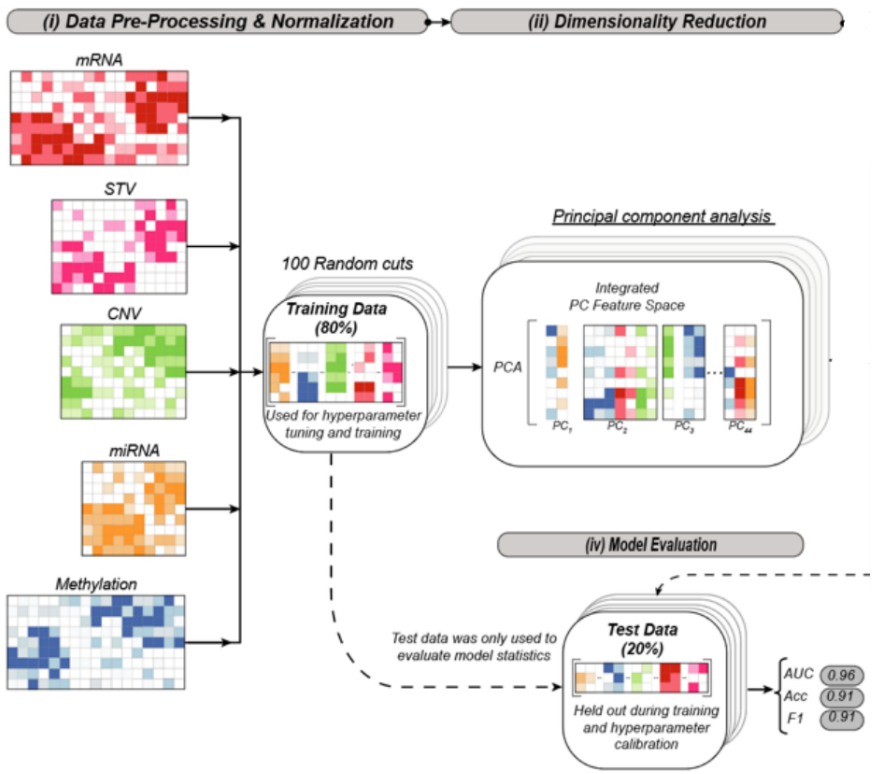

Figure 2: (i) Multiomics data was retrieved, processed and concatenated into a single scaled 'omics' data matrix. It is then split into sets representing 80% and 20% of the data, serving as train and test sets. (ii) The dataset is now standardised and PCA is applied [3]

3 Principal Component Analysis

The dataset which we used and described in the previous section was a high-dimensional dataset and it is impractical to train machine learning algorithms on a dataset of such high dimensions, therefore we used classical preprocessing to reduce the number of dimensions as well as make the data suitable for training on both classical and quantum SVM algorithms.

High dimensional datasets suffer from a phenomenon called the "curse of dimensionality". High dimensional datasets are at risk of being extremely sparse and most training instances are likely to be far away from each other. This implies that any new instance which we would want to predict would lie far away from any instance of a particular class, making the predictions extremely unreliable than the ones in lower dimensions. This also leads to the phenomenon of overfitting.

One approach to deal with this problem is to increase the number of training instances. Unfortunately, the training time increases exponentially with the number go dimensions, so adding more instances would make the model impractical to train. Therefore, the most commonly used technique is that of dimensionality reduction using projection. The most commonly used algorithm for this purpose is Principal Component Analysis, or PCA.

PCA first identifies the hyperplane that lies closest to the data, and then projects the data onto it. But before projecting the data, it is important that we choose the right hyperplane to project onto. One reasonable metric to make this decision is the variance of the data. One could select an axis that preserves the maximum amount of variance, as it is less likely to result in loss of information. PCA implements a slightly modified version of this idea: it chooses an axis such that the mean squared distance between the original data and the projected data on that axis is minimised [4].

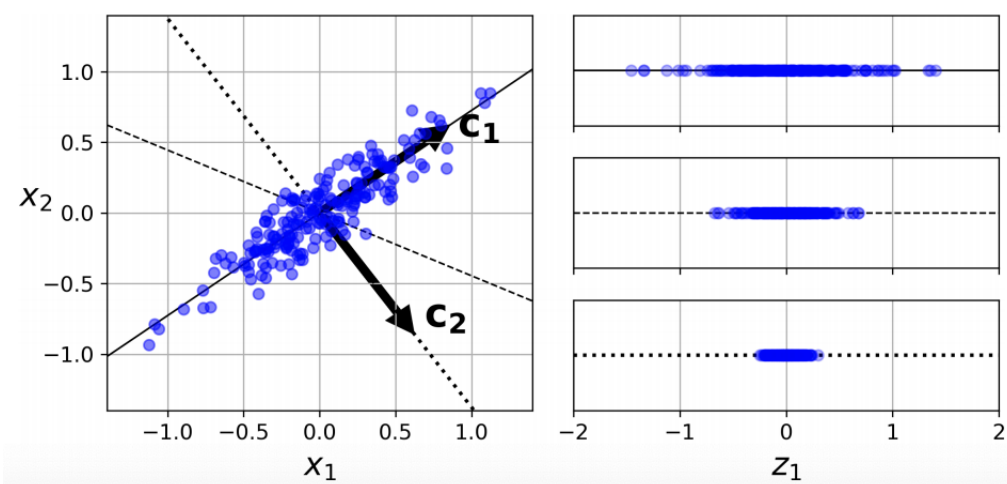

Figure 3: Projection of data along multiple axes

As seen in the above figure, a projection along the solid line preserves the maximum variance while a projection along the dotted line preserves very little.

3.1 Principal Components

PCA identifies the axis that accounts for the largest amount of variance in the training set. It also finds a second axis, orthogonal to the first one, that accounts for the largest amount of remaining variance. Next, it finds a third axis, orthogonal to the previous two axes, and a fourth, then a fifth, and so on — as many

axes as required by the user. The unit vector along the ith axis is called the ith Principal Component. The principal components of a matrix can be calculated using the technique of Singular Value Decomposition. Here the training matrix $\mathbf{X}$ is decomposed into a multiplication of three matrices $\mathbf{U\Sigma V}^T$, where $\mathbf{V}$ contains all the principal components that are required, as column vectors.

Once the principal components are determined, the dimensionality of the training set can be reduced down to d-dimensions by projecting it onto the hyperplane determined by the first d principal components. If $\mathbf{W}_d$ represents the matrix of the first d principal components, the reduced dataset is given by $\mathbf{X}_d = \mathbf{XW}_d$ [4] In Scikit-Learn, PCA is implemented using the `PCA` method that takes *n_components* as an argument that allows you to reduce the dataset to that many dimensions. It also provides for the user to specify the variance ratio to be preserved by setting the *n_components* parameter to a number between 0 and 1.

In our case, we decided to reduce the number of dimensions to 10 from the original 66,801. Although this represented a huge loss in information, we show in following sections that this did not affect our results much and that our resultant accuracies on the model were close to what was reported by the paper we consulted.

4 Feature Scaling

On of the most important classical preprocessing that is applied on data is feature scaling. Machine learning algorithms are sensitive to feature scales and perform rather poorly (of course, with some exceptions like Random Forests) when the numerical attributes have different scales. Feature scaling is done to ensure that all attribute conform to the same numerical scale.

This is done using tow methods: min-max scaling and standardisation. Min-max scaling is as follows: for each feature, values are shifted ans rescaled such that they lie between a certain range, usually 0 and 1. This is done by subtracting the minimum value of that feature from the point and dividing by the maximum of that feature. This can be implemented using Scikit-Learn's `MinMaxScaler` method.

Standardisation, on the other hand, involved a similar technique. Here, the mean value is subtracted ans then divided by the standard deviation of the feature values. This results in a distribution with zero mean and unit variance. This can be implemented using Scikit-Learn's `StandardScaler` method. Our classical preprocessing of the dataset involved standardising the dataset, then applying PCA to reduce the dimensions to 10, and finally using `MinMaxScaler` to resize the data to between -1 and 1.

5 Quantum Support Vector Machine

To understand the working of a SVM algorithm, it is imperative to consider the working of a classical SVM algorithm — the Least Squares SVM.

5.1 Least Squares SVM

A Support Vector Machine (SVM) is a commonly used supervised machine learning algorithm. Consider a set of independent and identically distributed training set $\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_i, y_i), \ldots, (\mathbf{x}_M, y_M)\}$, where $\mathbf{x}_i \subset \mathbb{R}^N$ and $y_i \in \{+1, -1\}$. The vector $\mathbf{x}_i$ is called the feature vector for the i-th training instance and y_i is a binary class label. Here, N is the dimension of the feature vectors and M is the number of training instances. The SVM algorithm, essentially, models a hyperplane that best fits the given data points such that the two classes are separated by a maximum margin [4]. For a given set of weights, $\mathbf{w}$, and an offset parameter b, the prediction of the SVM algorithm is given by

$$\hat{y}_i = \operatorname{sgn}\left(\mathbf{w}^T \mathbf{x}_i + b\right) \tag{5}$$

The margin of the SVM is defined as the distance of this hyperplane to the nearest positive or negative example (called the support vector). An important feature of the SVM algorithm is that it allows for a kernel mapping from the input space to a different feature space (For example: the Gaussian RBF kernel), given by an embedding function $\phi(\mathbf{x})$. The slope of the decision function is equal to the norm of the weight vector, $\|\mathbf{w}\|$, and to get a large enough margin, we must minimise $\|\mathbf{w}\|$. If we also introduce a parameter ζ_i, which measures the allowed margin violation for the instance $\mathbf{x}_i$,; and also a parameter C which allows to determine the tradeoff between minimising ζ_i to minimise margin violations and minimising $\frac{1}{2}\|\mathbf{w}\|^2 = \frac{1}{2}\mathbf{w}^T\mathbf{w}$ (reformulation for easier gradient calculation) to increase the margin, the objective function for the SVM is

$$\min_{\mathbf{w}, b, \zeta} \quad \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{M} \zeta_i \qquad \text{Subject to } y_i(\mathbf{w}^T\phi(\mathbf{x}_i) + b) \geq 1 - \zeta_i \tag{6}$$

Practically, however, the dual optimisation problem for the primal problem is solved by introducing Lagrangian multipliers [4], which is:

$$\max \quad \sum_{i=1}^{M} \alpha_i - \frac{1}{2}\sum i = 1^M \sum_{j=1}^{m} \alpha_i y_i \alpha_j y_j K(\mathbf{x}_i, \mathbf{x}_j) \tag{7}$$

Subject to

$$\sum_{i=1}^{M} \alpha_i y_i = 0, \quad \alpha_i \in [0, C], \quad i = 1, 2, \ldots, M$$

$K(\mathbf{x}_i, \mathbf{x}_j)$ is called the kernel function: the dot product of the embedding space. Therefore the new decision function is:

$$\hat{y}_i = \operatorname{sgn}\left(\sum_{i=1}^{M} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b\right) \tag{8}$$

Least squares SVMs modify the objective function of the primal problem by using the ℓ^2-norm in the regularisation term [5]:

$$\min \quad \frac{1}{2}\mathbf{w}^T\mathbf{w} + \frac{\gamma}{2}\sum_{i=1}^{M} e_i^2 \qquad \text{Subject to } y_i(\mathbf{w}^T\phi(\mathbf{x}_i) + b) = 1 - e_i \tag{9}$$

γ has the same function as C. The saddle point of the corresponding Lagrangian is given by the linear equation:

$$\begin{pmatrix} 0 & \mathbb{1}^T \\ \mathbb{1} & K + \gamma^{-1}\mathbb{1} \end{pmatrix} \begin{pmatrix} b \\ \alpha \end{pmatrix} = \begin{pmatrix} 0 \\ \mathbf{y} \end{pmatrix} \tag{10}$$

Where $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$. For a linear or polynomial kernel, classically, calculation of entries the kernel matrix takes $\mathcal{O}(N)$ time, and therefore the whole matrix takes $\mathcal{O}(M^2N)$ time. Solving the dual problem takes $\mathcal{O}(M^3)$ time. Combining the two, we obtain that the classical SVM takes $\mathcal{O}(M^2(M+N))$ time.

5.2 Working of QSVM

The QSVM uses a quantum approach towards calculating the kernel matrix, thus speeding up the SVM algorithm [6, 7]. Here, the training instances are presented as quantum states $|x_i\rangle$. They need not be normalised, but the normalisation must be given separately. To find a dot product of two training instances, here two states (for the kernel matrix), we use the approach given by Lloyd et al. [8]:

- Generate two states, $|\psi\rangle$ and $|\phi\rangle$, with an ancillary variable;
- Estimate the parameter $Z = |\mathbf{x}_i|^2 + |\mathbf{x}_j|^2$ — The sum of the squared norms of the two instances;
- Perform a projective measurement on the ancillary alone, comparing the two states.

Z times the probability of success of the measurement yields the square of the Euclidean distance between the two training instances: $|\mathbf{x}_i - \mathbf{x}_j|^2$. The dot product in the linear kernel, therefore, is

$$\mathbf{x}_i^T \mathbf{x}_j = \frac{Z - |\mathbf{x}_i - \mathbf{x}_j|^2}{2} \tag{11}$$

The two states are usually

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle\,|\mathbf{x}_i\rangle + |1\rangle\,|\mathbf{x}_j\rangle)$$

and

$$|\phi\rangle = \frac{1}{\sqrt{2}}(|0\rangle\,|\mathbf{x}_i| + |1\rangle\,|\mathbf{x}_j|)$$

To produce $|\phi\rangle$, we evolve the state $|-\rangle\,|0\rangle$ with the Hamiltonian $H = (|\mathbf{x}_i|\,|0\rangle\,\langle 0| + |\mathbf{x}_j|\,|1\rangle\,\langle 1|) \otimes \sigma_x$, which results in the state $\frac{1}{\sqrt{2}}(\cos(|\mathbf{x}_i|t)\,|0\rangle - \cos(|\mathbf{x}_j|t)\,|1\rangle)\,|0\rangle - \frac{i}{\sqrt{2}}(\sin(|\mathbf{x}_i|t)\,|0\rangle - \sin(|\mathbf{x}_j|t)\,|1\rangle)\,|0\rangle$. By choosing an appropriate choice of t, and measuring the ancilla, we obtain the state $|\phi\rangle$ with probability $\frac{1}{2}Z^2t^2$, which allows us to estimate Z. If the error required in measurement is ϵ, construction of $|\phi\rangle$ is done in $\mathcal{O}(\epsilon^{-1})$. Giovannetti et al. [9] say that the time required to construct a state from QRAM is $\mathcal{O}(\log N)$. Therefore, the time required to estimate the dot product $\mathbf{x}_i^T\mathbf{x}_j$ is $\mathcal{O}(\epsilon^{-1}\log N)$. Finally, for the entire algorithm, the time complexity is now $\mathcal{O}(M^2(M + \epsilon^{-1}\log N))$

Therefore, we have justified how a quantum approach towards the SVM provides an exponential speedup to the classical least-squares SVM algorithm.

6 Methods and Experiments

We utilised the Qiskit-Aqua framework to implement both — the classical least-squares SVM and the quantum SVM. We used a local computer as the backend for the quantum simulator. We applied two machine earning algorithms: Quantum SVM and Classical SVM. These were accessed through the `QSVM()` and `SklearnSVM()` methods. We ran the algorithms in the IPynb environment provided by Jupiter Notebook for ease of access. The classical SVM is a SVM Radial Basis Function (RBF) Kernel SVM supplied with Aqua. We also used two types of backends: (1) `qasm_simulator` for noise analysis; and (2) `statevector_simulator` for quantum simulation. The metric we used for the comparison was the balanced accuracy since the dataset was highly biased towards one class.

We aimed to do three experiments. First, we wanted to study how both ML algorithms performed on the entire dataset. Next, we wanted to see if any algorithm would offer a significant advantage when the size of the training set was reduced, and finally, we wanted to study the effects of noise on the balanced accuracy.

7 Results and Analysis

We initially split the dataset into training (80%) and test (20%) datasets. We then fit both algorithms on the training set and computed the balanced accuracy on the test set 5 times and computed the average balanced accuracy and the standard error of the mean. The results obtained were as follows:

Table 1: Quantum SVM results on the LumA vs LumB datasets

Classifier	Backend	Balanced Accuracy
Quantum SVM	`statevector_simulator`	0.651 ± 0.012
Classical SVM	`statevector_simulator`	0.635 ± 0.009

As seen, the quantum SVM marginally better than the classical SVM on the entire dataset, although the performance is quite poor. This was to be expected, as quantum algorithms perform better on smaller number of training instances due to their tendency to overfit the data.

Next we ran the same experiment on an actual quantum processor using IBM's cloud experience. The results obtained are tabulated below:

Table 2: Quantum SVM results on the LumA vs. LumB datasets

Classifier	Backend	Balanced Accuracy
Quantum SVM	`ibm_16_melbourne`	0.607
Classical SVM	`ibm_16_melbourne`	0.624

It is observed that the performance has dropped. This was because an actual quantum processor introduced noise in the execution that manifests as a reduction the the balanced accuracy.

Now, we ran the algorithms on a fraction of the original training dataset. This was chaired by randomly sampling a certain percentage of the training data using Scikit-Learn's `train_test_split` method and setting its `train_size` parameter to the required value. Post training, both models were evaluated on the same test dataset as before. The backend used was `statevector_simulator`. The results obtained are tabulated below:

Table 3: Quantum SVM results on the LumA vs LumB datasets, Fraction of Training Data

Fraction of the training dataset	Classical Score	Quantum Score
100%	0.635	0.651
90%	0.652	0.638
80%	0.683	0.642
70%	0.711	0.667
60%	0.679	0.654
50%	0.661	0.641
40%	0.577	0.651
30%	**0.577**	**0.705**
20%	0.610	0.698
10%	0.523	0.615

We see form the above data that the quantum support vector machine performs exceedingly well when it is trained on 30% of the original training data (*i.e*, 75 training instances). This is also in agreement with the results obtained by Li et al. and a few others, where they claimed that quantum-based approaches were superior to classical classification methods on small training set sizes [3, 10, 11, 12].

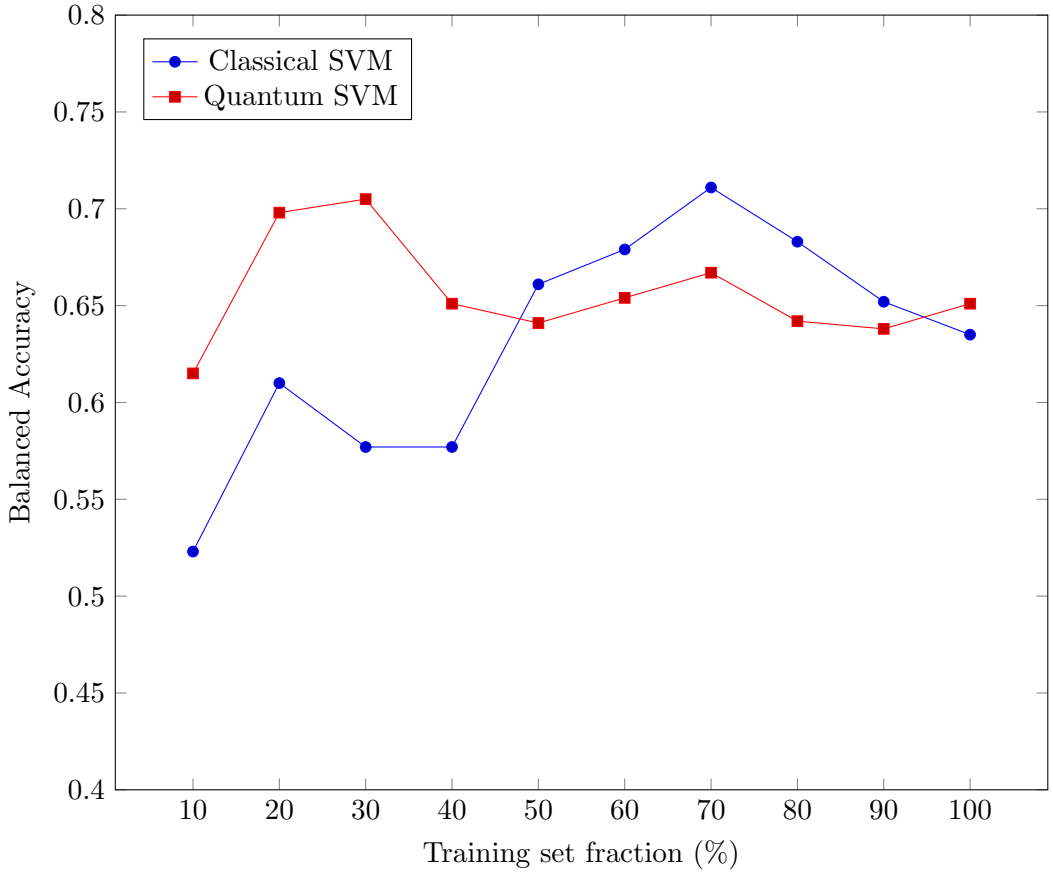

Figure 4: Graph between balanced accuracy and training set fraction for classical and quantum classifiers

It was observed that on average on the whole dataset, QSVM — with an accuracy of 65.1% — performed marginally better that classical SVM — which had an accuracy of 63.5%. One point to be noted is the low accuracy which we obtained. This is due to the large loss of information because we applied principal component reduction on the data to reduce it to 10 dimensions, from 66,801. This corresponds to a decrease of almost 99%, which is huge. This was inevitable as quantum approaches require you to have as many dimensions as the number of qubits, which in this case was 16. The remaining 6 qubits would be used for error correction. We gained a few key insights after reviewing these results. First of all, fitting quantum SVM models on large datasets is a computationally expensive task. This can be improved by running the program on an actual quantum computer — physically and not through the cloud. Second, this performance of quantum classifiers is not always superior to classical ones and is highly dependent on the dataset. Therefore, if one chooses to use a large dataset it would be much more efficient to use classical methods over quantum methods (although QSVMs provided a larger accuracy) due to the large run times involved with a quantum classifier.

The final result we obtained does indicate that quantum classifiers performed better than their classical counterparts, but this is only a comparative study to show a proof-of-concept. It is unknown if this performance can be consistently replicated a large number of times.

Therefore, to summarise our work, we performed a comparative analysis of the quantum and classical support vector machine algorithms, and we would propose that quantum methods offer a **significant advantage over classical methods**, but only **conditionally**. The performance is linked and correlated to the classification task in question as well as the nature of the hardware involved. We do, however, see an increase in the acceptability and ubiquity of quantum methods in the near future with advancements in quantum hardware and algorithms.

8 Bibliography

[1] R. Vathsan, *Introduction to quantum physics and information processing* (CRC Press, 2015).

[2] M. A. Nielsen and I. Chuang, *Quantum computation and quantum information* (American Association of Physics Teachers, 2002).

[3] R. Y. Li, S. Gujja, S. R. Bajaj, O. E. Gamel, N. Cilfone, J. R. Gulcher, D. A. Lidar, and T. W. Chittenden, arXiv preprint arXiv:1909.06206 (2019).

[4] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems* (O'Reilly Media, 2019).

[5] J. A. Suykens and J. Vandewalle, Neural processing letters **9**, 293 (1999).

[6] P. Rebentrost, M. Mohseni, and S. Lloyd, Physical review letters **113**, 130503 (2014).

[7] P. Wittek, *Quantum machine learning: what quantum computing means to data mining* (Academic Press, 2014).

[8] S. Lloyd, M. Mohseni, and P. Rebentrost, arXiv preprint arXiv:1307.0411 (2013).

[9] V. Giovannetti, S. Lloyd, and L. Maccone, Physical review letters **100**, 160501 (2008).

[10] R. Y. Li, R. Di Felice, R. Rohs, and D. A. Lidar, NPJ quantum information **4**, 1 (2018).

[11] A. Mott, J. Job, J.-R. Vlimant, D. Lidar, and M. Spiropulu, Nature **550**, 375 (2017).

[12] D. Willsch, M. Willsch, H. De Raedt, and K. Michielsen, Computer Physics Communications **248**, 107006 (2020).

9 Glossary

- **Accuracy:** The ratio of correctly predicted classes to the total number classes predicted. Lies between 0 and 1.
- **Algorithm:** A sequence of finite, well-defined instruction designed to solve a particular problem or achieve a given task.
- **Balanced Accuracy:** The balanced accuracy is a metric used in in binary and multiclass classification problems to deal with imbalanced datasets. It is defined as the average of recall obtained on each class.
- **Big-O notation:** Seen as $\mathcal{O}()$. It is a mathematical notation that describes the limiting behaviour of a function when its argument approaches large values or infinity.
- **Cost Function:** A function that maps a possible solution to a problem to a numerical value to represent the "cost" associated with choosing that option as the answer.
- **Dataset:** A coherent collection of data. Serves as the source for training statistical models.
- **Dirac Notation :** A notation invented by Paul Dirac to represent vectors of a Hilbert space. The ket (or $|a\rangle$) (represented by a column vector) represents the vector belonging to the abstract vector space and the bra (or $\langle a|$) (represented by a row vector) represents a linear functional on that space, or a element of the corresponding dual vector space.
- **Features:** A measurable piece of data that can be used for analysis. Represented as a column in a dataset.
- **Feature Selection:** A process of choosing those features which are required to explain the predictive power of a statistical model and dropping out irrelevant features.
- **Hamiltonian:** A quantum mechanical operator corresponding to the sum of kinetic energy and potential energy of the system. Usually denoted by H or $\hat{H}$.
- **Hilbert Space:** A complete vector space with a defined inner product. In quantum mechanics, this inner product is usually the Euclidean norm of the vector. Represented by $\mathcal{H}$
- **Kernel:** (In SVM) A map between two (feature) vectors and the inner product between their embedding function values.
- **Lagrange Multipliers:** A optimisation technique to find the extremum values of a function subject to equality constraints.
- **NP Problem:** Stands for Nondeterministic Polynomial-time problem. Set of problems whose solution can be verified in polynomial time.
- **Pauli Operators:** A set of 3 (4 if $\mathbb{1}$ is counted) complex, Hermitian and unitary matrices $\{\sigma_x, \sigma_y, \sigma_z\}$; fundamental in quantum mechanics and represent the observable corresponding to a spin in the x, y, or z direction.
- **Qubit:** A two-state quantum system. Fundamental unit of quantum computation.